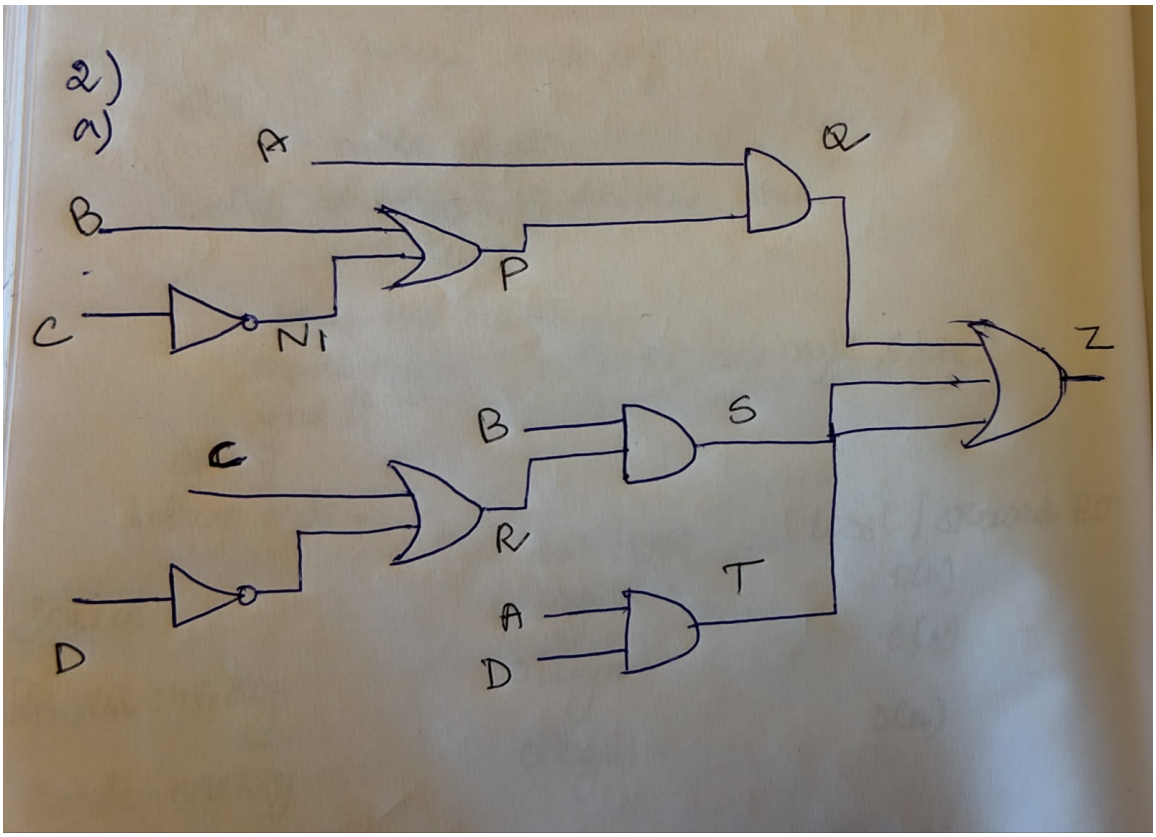
Karishma Premnath
EECE 7205, Stacks and Queues, In-Class Work
Q2 Logic Gate Analysis and Implementation

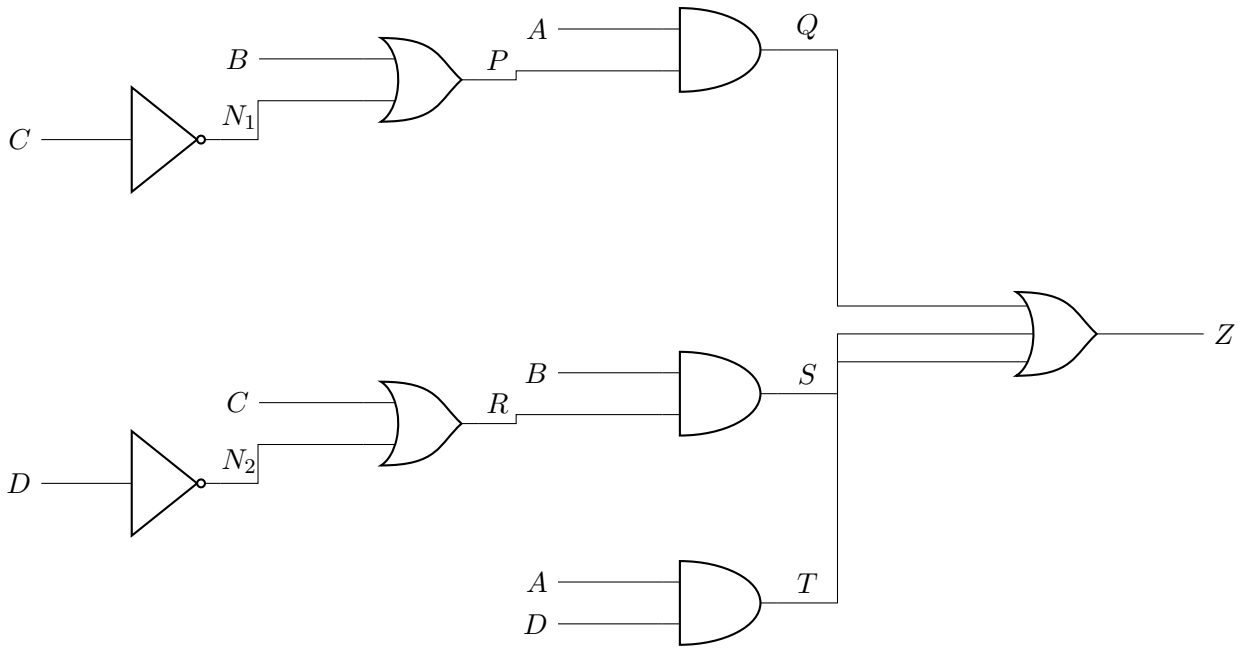
(a) Circuit and intermediate signals

Four levels, eight gates:

Level	Gate	Inputs	Output label
1	NOT	C	N_1
1	NOT	D	N_2
2	OR	B, N_1	P
2	OR	C, N_2	R
3	AND	A, P	Q
3	AND	B, R	S
3	AND	A, D	T
4	OR (3 input)	Q, S, T	Z

Each input is used twice: A feeds Q and T, B feeds P and S, C feeds N_1 and R, D feeds N_2 and T.





(b) Boolean expression

Working through the gate list in order:

$$
\begin{aligned}
N_1 &= C' \\
N_2 &= D' \\
P &= B + C' \\
Q &= A(B + C') \\
R &= C + D' \\
S &= B(C + D') \\
T &= AD \\
Z &= Q + S + T = A(B + C') + B(C + D') + AD
\end{aligned}
$$

Multiplied out:

$$Z = AB + AC' + BC + BD' + AD$$

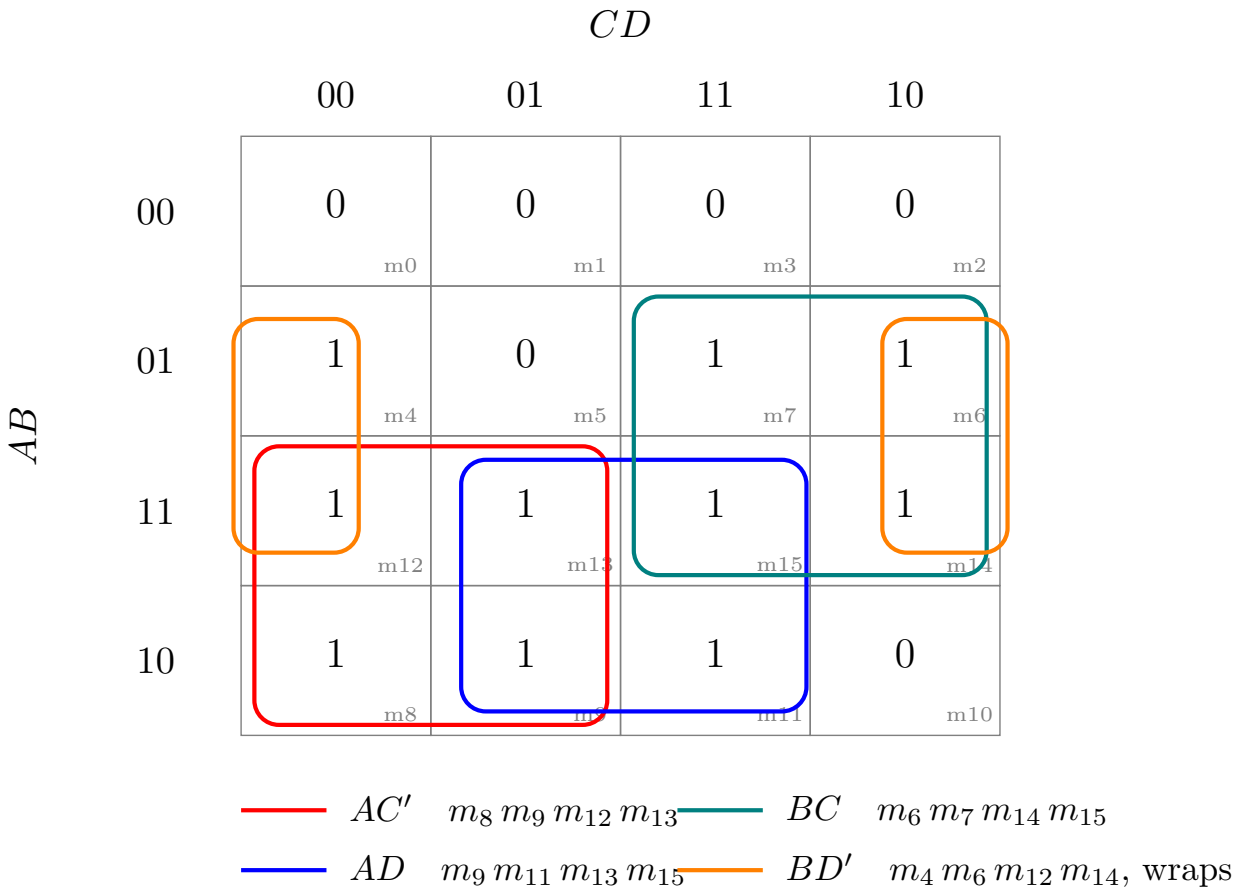
(c) Truth table

A	B	C	D	N_1	N_2	P	Q	R	S	T	Z
0	0	0	0	1	1	1	0	1	0	0	**0**
0	0	0	1	1	0	1	0	0	0	0	**0**
0	0	1	0	0	1	0	0	1	0	0	**0**
0	0	1	1	0	0	0	0	1	0	0	**0**
0	1	0	0	1	1	1	0	1	1	0	**1**
0	1	0	1	1	0	1	0	0	0	0	**0**
0	1	1	0	0	1	1	0	1	1	0	**1**
0	1	1	1	0	0	1	0	1	1	0	**1**
1	0	0	0	1	1	1	1	1	0	0	**1**
1	0	0	1	1	0	1	1	0	0	1	**1**
1	0	1	0	0	1	0	0	1	0	0	**0**
1	0	1	1	0	0	0	0	1	0	1	**1**
1	1	0	0	1	1	1	1	1	1	0	**1**
1	1	0	1	1	0	1	1	0	0	1	**1**
1	1	1	0	0	1	1	1	1	1	0	**1**
1	1	1	1	0	0	1	1	1	1	1	**1**

$Z = 1$ for ten input combinations: m_4, m_6, m_7, m_8, m_9, m_{11}, m_{12}, m_{13}, m_{14}, m_{15}.

(d) Karnaugh map and minimum SOP

Rows AB and columns CD, both in Gray-code order. The four groups are drawn on the map:



Four groups of four cells:

Group	Cells	Term
$B = 1$, $D = 0$	m_4, m_6, m_{12}, m_{14}	BD'
$B = 1$, $C = 1$	m_6, m_7, m_{14}, m_{15}	BC
$A = 1$, $D = 1$	m_9, m_{11}, m_{13}, m_{15}	AD
$A = 1$, $C = 0$	m_8, m_9, m_{12}, m_{13}	AC'

$$\boxed{Z = BD' + BC + AD + AC'}$$

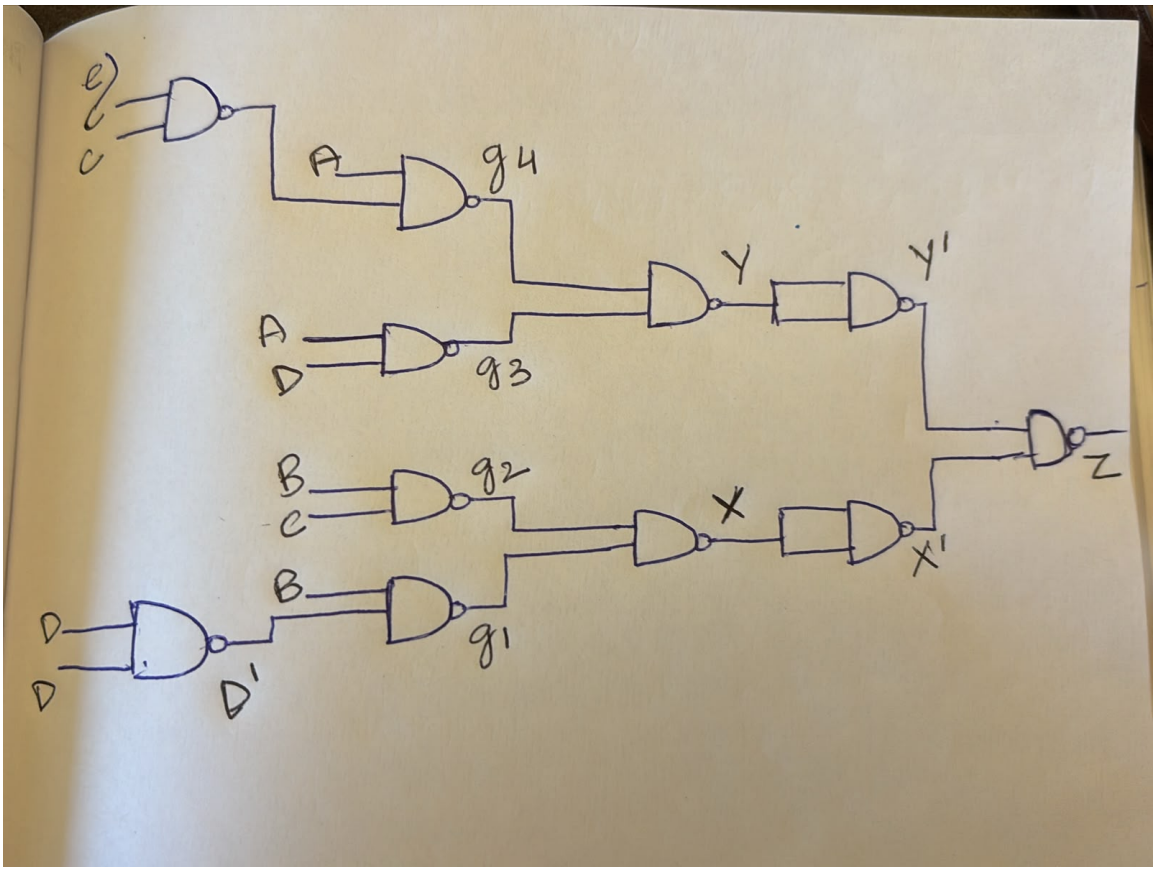
The BD' group wraps around the left and right edges of the map, since the $CD = 00$ and $CD = 10$ columns are adjacent in Gray-code order. That is why it appears as two halves.

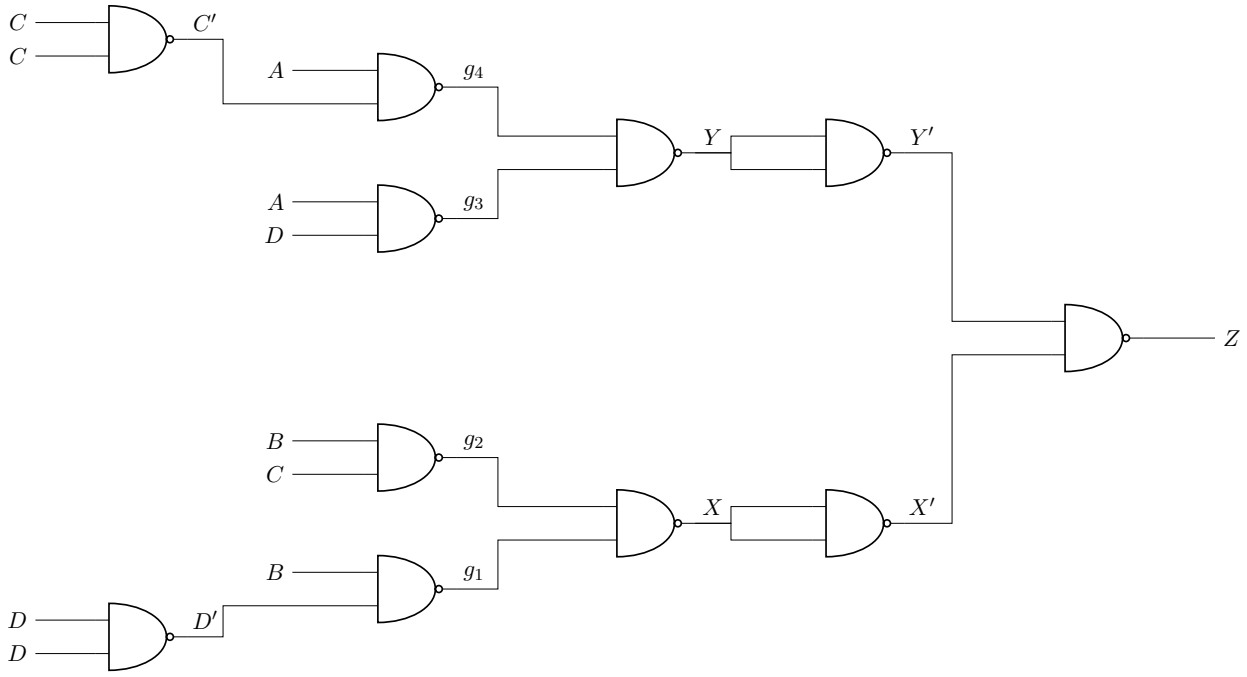
AB is also a prime implicant, covering m_{12}, m_{13}, m_{14}, m_{15}, but it is redundant because all four of those cells are already covered by the groups above. The four-term cover is the only minimum one.

(e) NAND-only implementation

Starting from $Z = BD' + BC + AD + AC'$. Each product becomes a NAND, the sum becomes a NAND of the inverted partial sums. A NAND with both inputs tied together acts as an inverter. Eleven two-input NAND gates:

Gate	Inputs	Output	Meaning
1	C, C	C'	inverter
2	D, D	D'	inverter
3	B, D'	g_1	$(BD')'$
4	B, C	g_2	$(BC)'$
5	A, D	g_3	$(AD)'$
6	A, C'	g_4	$(AC')'$
7	g_1, g_2	X	$BD' + BC$
8	g_3, g_4	Y	$AD + AC'$
9	X, X	X'	inverter
10	Y, Y	Y'	inverter
11	X', Y'	Z	$X + Y$





Checked against the original circuit for all sixteen input combinations, the outputs agree on every row.

(f) Worst-case propagation delay

Every NAND gate has a delay of 20 ns, so the delay of a path is 20 ns times the number of gates on it.

Path	Gates	Delay
$C \to C' \to g_4 \to Y \to Y' \to Z$	5	100 ns
$D \to D' \to g_1 \to X \to X' \to Z$	5	100 ns
$A \to g_3 \to Y \to Y' \to Z$	4	80 ns
$B \to g_2 \to X \to X' \to Z$	4	80 ns

The longest input-to-output path is five gates, so the worst-case propagation delay is

$$5 \times 20 \text{ ns} = 100 \text{ ns}$$

reached from inputs C and D, because those two have to pass through an inverter before reaching a product gate. Inputs A and B are used uncomplemented and reach the output in four gates.

Evaluating the expression with a stack

The minimised expression is a sum of products, so it needs no parentheses and runs on the same two-stack evaluator as Q1. The values stack holds 0 and 1, the operators stack holds `&` and `|`, and `&` has higher precedence than `|`. A token like `C'` is read as the complement of C.

```
part (b):  A & B | A & C' | B & C | B & D' | A & D
part (d):  B & D' | B & C | A & D | A & C'
```

The program evaluates both the part (b) expression and the part (d) minimised expression for all sixteen input combinations, and compares each against the original gate network.

```
// Q2: evaluate the circuit expression with two stacks, same method as Q1.
// Values stack holds 0/1, operators stack holds & and |.  & binds tighter than |.
// A token like C' means the complement of C.

#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
using namespace std;

const int CAPACITY = 100;

class IntStack
{
private:
    int data[CAPACITY];
    int top = -1;
public:
    bool isEmpty() const { return top == -1; }
    bool isFull() const { return top == CAPACITY - 1; }
    void push(int x) { if (!isFull()) { top++; data[top] = x; } }
    int pop() { int x = data[top]; top--; return x; }
};

class CharStack
{
private:
    char data[CAPACITY];
    int top = -1;
public:
    bool isEmpty() const { return top == -1; }
    bool isFull() const { return top == CAPACITY - 1; }
    char peek() const { return data[top]; }
    void push(char c) { if (!isFull()) { top++; data[top] = c; } }
    char pop() { char c = data[top]; top--; return c; }
};

bool isVariable(const string &t)
{
    return t[0] >= 'A' && t[0] <= 'D';
}

int valueOf(const string &t, int A, int B, int C, int D)
{
    int v = 0;
    if (t[0] == 'A') v = A;
    else if (t[0] == 'B') v = B;
```

```
    else if (t[0] == 'C') v = C;
    else v = D;
    if (t.size() > 1 && t[1] == '\'') v = 1 - v;
    return v;
}

int precedence(char op) { return (op == '&') ? 2 : 1; }

int applyOp(int a, int b, char op) { return (op == '&') ? (a & b) : (a | b); }

void applyTop(IntStack &values, CharStack &ops)
{
    char op = ops.pop();
    int b = values.pop();
    int a = values.pop();
    values.push(applyOp(a, b, op));
}

int evaluate(const vector<string> &tokens, int A, int B, int C, int D)
{
    IntStack values;
    CharStack ops;

    for (int i = 0; i < (int)tokens.size(); i++)
    {
        if (isVariable(tokens[i]))
        {
            values.push(valueOf(tokens[i], A, B, C, D));
            continue;
        }
        char op = tokens[i][0];
        while (!ops.isEmpty() && precedence(ops.peek()) >= precedence(op))
            applyTop(values, ops);
        ops.push(op);
    }
    while (!ops.isEmpty())
        applyTop(values, ops);

    return values.pop();
}

int main()
{
    vector<string> Zb   = {"A", "&", "B", "|", "A", "&", "C'", "|", "B", "&", "C",
                           "|", "B", "&", "D'", "|", "A", "&", "D"};
    vector<string> Zmin = {"B", "&", "D'", "|", "B", "&", "C",
                           "|", "A", "&", "D", "|", "A", "&", "C'"};

    cout << " A B C D | N1 N2  P  Q  R  S  T |  Z gates  Z (b)  Z (d)" << endl;
    cout << "---------+----------------------+----------------------" << endl;

    bool allMatch = true;
    for (int A = 0; A <= 1; A++)
      for (int B = 0; B <= 1; B++)
        for (int C = 0; C <= 1; C++)
          for (int D = 0; D <= 1; D++)
          {
              int N1 = !C, N2 = !D;
              int P = B | N1, Q = A & P;
```

```
            int R = C | N2, S = B & R;
            int T = A & D;
            int Zgates = Q | S | T;

            int Zexpanded = evaluate(Zb, A, B, C, D);
            int Zminimal  = evaluate(Zmin, A, B, C, D);
            if (Zgates != Zexpanded || Zgates != Zminimal) allMatch = false;

            cout << " " << A << " " << B << " " << C << " " << D << " |"
                 << setw(3) << N1 << setw(3) << N2 << setw(3) << P
                 << setw(3) << Q  << setw(3) << R  << setw(3) << S
                 << setw(3) << T  << " |" << setw(6) << Zgates
                 << setw(8) << Zexpanded << setw(7) << Zminimal << endl;
        }

    cout << endl
         << "Z (b) = A & B | A & C' | B & C | B & D' | A & D" << endl
         << "Z (d) = B & D' | B & C | A & D | A & C'" << endl << endl
         << "both expressions match the gate circuit on all 16 rows: "
         << (allMatch ? "yes" : "no") << endl;
    return 0;
}
```

Output

```
 A B C D | N1 N2  P  Q  R  S  T |  Z gates  Z (b)  Z (d)
---------+----------------------+---------------------
 0 0 0 0 |  1  1  1  0  1  0  0 |     0       0      0
 0 0 0 1 |  1  0  1  0  0  0  0 |     0       0      0
 0 0 1 0 |  0  1  0  0  1  0  0 |     0       0      0
 0 0 1 1 |  0  0  0  0  1  0  0 |     0       0      0
 0 1 0 0 |  1  1  1  0  1  1  0 |     1       1      1
 0 1 0 1 |  1  0  1  0  0  0  0 |     0       0      0
 0 1 1 0 |  0  1  1  0  1  1  0 |     1       1      1
 0 1 1 1 |  0  0  1  0  1  1  0 |     1       1      1
 1 0 0 0 |  1  1  1  1  1  0  0 |     1       1      1
 1 0 0 1 |  1  0  1  1  0  0  1 |     1       1      1
 1 0 1 0 |  0  1  0  0  1  0  0 |     0       0      0
 1 0 1 1 |  0  0  0  0  1  0  1 |     1       1      1
 1 1 0 0 |  1  1  1  1  1  1  0 |     1       1      1
 1 1 0 1 |  1  0  1  1  0  0  1 |     1       1      1
 1 1 1 0 |  0  1  1  1  1  1  0 |     1       1      1
 1 1 1 1 |  0  0  1  1  1  1  1 |     1       1      1

Z (b) = A & B | A & C' | B & C | B & D' | A & D
Z (d) = B & D' | B & C | A & D | A & C'

both expressions match the gate circuit on all 16 rows: yes
```